

REACT COURSE

Lesson 1 — React Introduction & Setups

React Basics • SPA • Vite • JSX • Project Structure

Course	React Development
Lesson	01
Level	Beginner
Focus	React Introduction & Setup

React Course — Lesson 1

1. Topic

React Introduction & Setups

What is React?

Why do we need React?

React features

Single Page Application (SPA) concept

React vs Vanilla JavaScript

Node.js and npm basics

Creating a React project with Vite

React folder structure

JSX introduction

JSX expressions

JSX rules

2. Definition

React is an open-source JavaScript library used for building fast, interactive, and dynamic user interfaces. It is maintained by Meta and supported by a large global community of developers. React helps developers build modern web applications by dividing the user interface into small, reusable pieces called components.

React mainly focuses on the frontend or user interface of web applications. It is especially useful for applications where data changes frequently and the interface needs to respond to those changes.

3. What is React?

React is a JavaScript library for creating user interfaces. A user interface contains all the parts of an application that users see and interact with, such as buttons, forms, navigation bars, cards, dashboards, tables and profiles.

React uses a component-based approach. Instead of creating one large page, developers divide the UI into smaller components. Each component can have its own logic and can be reused in different parts of an application.

For example, an LMS application may contain:

Navbar

Sidebar

Course Card

Lesson List

Quiz

Progress Bar

Student Profile

4. Why Do We Need React?

In a small website, HTML, CSS and normal JavaScript may be enough. But when an application becomes large, manually selecting DOM elements and keeping many UI parts synchronized can become difficult.

React helps solve these problems by providing:

Reusable components

Automatic UI updates when state changes

Better organization of large applications

Declarative UI development

Efficient UI updates

Easy integration with other libraries and APIs

5. React Features

Feature	Meaning
Component Based	UI is divided into small reusable components.
Reusable Components	The same component can be used with different data.
JSX	HTML-like syntax can be written inside JavaScript.
Declarative UI	Developers describe what the UI should look like.
Virtual DOM	React efficiently manages changes to the UI.
One-Way Data Flow	Data generally moves from parent components to child components.
Large Ecosystem	Many tools and libraries are available around React.

6. Single Page Application (SPA)

SPA stands for Single Page Application. In an SPA, the application can update the required content without loading a completely new HTML page for every user action.

Traditional website flow:

User → Request → Server → New Page → Browser Reload

SPA flow:

User → React Application → Data/UI Update → Required Part Changes

Examples include dashboards, LMS platforms, e-commerce interfaces and social media applications. The main benefit is a smoother and more app-like user experience.

7. React vs Vanilla JavaScript

Vanilla JavaScript means using JavaScript directly without a frontend library such as React.

Vanilla JavaScript Example

```
<button id="btn">Click Me</button>
<h1 id="heading">Hello</h1>
```

```
<script>
const button = document.getElementById("btn");
const heading = document.getElementById("heading");
```

```
button.addEventListener("click", () => {
  heading.textContent = "Button Clicked";
});
</script>
```

React Example

```
import { useState } from "react";
```

```
function App() {
  const [message, setMessage] = useState("Hello");
```

```
  return (
    <div>
```

```
<h1>{message}</h1>
<button onClick={() => setMessage("Button Clicked")}>
Click Me
</button>
</div>
);
}
```

```
export default App;
```

In the Vanilla JavaScript example, the developer directly selects and changes DOM elements. In React, the state controls the UI and React handles the required update.

8. Node.js and npm Basics

Node.js is a JavaScript runtime that allows JavaScript tools to run outside the browser. React development tools such as Vite use Node.js.

npm stands for Node Package Manager. It is used to install, update and manage packages required by a JavaScript project.

```
node -v
npm -v
npm install axios
```

The package.json file stores project information, scripts and dependencies. The node_modules folder contains installed packages.

9. Creating a React Project with Vite

Vite is a modern frontend build tool that makes creating and running React projects quick and simple.

Step 1: Create the project

```
npm create vite@latest
```

Step 2: Select the project name, React framework and JavaScript variant.

Step 3: Move into the project folder

```
cd my-react-app
```

Step 4: Install dependencies

```
npm install
```

Step 5: Start the development server

```
npm run dev
```

Vite will show a local development address, commonly similar to:

```
http://localhost:5173
```

React Course — Lesson 1

10. React Project Folder Structure

```
my-react-app/  
|  
├── node_modules/  
├── public/  
|  
├── src/  
|   ├── assets/  
|   ├── App.jsx  
|   ├── App.css  
|   ├── main.jsx  
|   └── index.css  
|  
├── .gitignore  
├── index.html  
├── package.json  
├── package-lock.json  
└── vite.config.js
```

Important Folders and Files

node_modules: Contains installed project packages.

public: Used for static files that can be served directly.

src: Contains the main React source code.

App.jsx: Commonly used as the main application component.

main.jsx: Connects the React application to the root HTML element.

package.json: Contains scripts, project information and dependencies.

package-lock.json: Stores the exact dependency versions used by npm.

index.html: The main HTML entry page.

vite.config.js: Configuration file for Vite.

11. JSX Introduction

JSX stands for JavaScript XML. It allows developers to write HTML-like syntax inside JavaScript code. JSX makes React components easier to read and write.

```
function App() {  
  return <h1>Hello React</h1>;  
}
```

```
export default App;
```

JSX is not exactly HTML. It is syntax that is transformed into JavaScript during the build process. Because JSX is part of JavaScript, JavaScript values and expressions can be inserted into it.

12. JSX Expressions

JavaScript expressions can be written inside JSX using curly braces { }. Variables, calculations, function calls and many other expressions can be used.

```
function App() {  
  const name = "Rahul";  
  const age = 20;  
  
  return (  
    <div>  
      <h1>Hello {name}</h1>  
      <p>Age: {age}</p>  
      <p>Next year: {age + 1}</p>  
    </div>  
  );  
}
```

```
export default App;
```

Array Example

```
const topics = ["JSX", "Components", "Setup"];
```

```
<p>{topics.join(", ")}</p>
```

13. JSX Rules

- 1. Single Parent Element:** Multiple JSX elements should be wrapped inside one parent element or fragment.
- 2. Close All Tags:** Every JSX element must be properly closed.
- 3. className:** Use className instead of the HTML class attribute.
- 4. Curly Braces:** Use { } to insert JavaScript expressions.
- 5. camelCase Attributes:** Many JSX event and property names use camelCase, such as onClick.
- 6. Comments:** JSX comments are written using {/ * comment */}.

14. Complete Example

A simple React component demonstrating JSX expressions, conditional rendering and list rendering:

```
const student = {  
  name: "John",  
  isEnrolled: true,  
  courses: ["React", "Node.js", "MongoDB"],  
};  
  
const element = (  
  <div>  
    <h1>Welcome, {student.name}!</h1>  
    {student.isEnrolled ? <p>You are enrolled</p> : <p>Please enroll</p>}  
    <ul>  
      {student.courses.map((course, index) => (  
        <li key={index}>{course}</li>  
      ))}  
    </ul>  
  </div>  
)
```



```
</ul>
</div>
);
```

15. Code Example

```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);
  const courseName = "React Introduction";
  const topics = ["JSX", "Components", "Setup"];

  return (
    <div>
      <h1>{courseName}</h1>
      <p>Topics: {topics.join(", ")}</p>
      <p>Count: {count}</p>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrement
      </button>
    </div>
  );
}

export default App;
```

16. Code Explanation

useState: The useState hook is imported to manage changing data inside the component.

count: The count variable stores the current state value.

setCount: setCount is used to update the count value.

courseName: Stores the course name and is displayed using a JSX expression.

topics: An array containing the names of lesson topics.

join(): Converts the topics array into a comma-separated string.

onClick: Runs the given function when a button is clicked.

Re-render: After state changes, React renders the component again with the updated state.

17. React Basic Flow

User Action



State Update



React Re-renders Component



UI Shows Updated Data

This simple flow is one of the most important ideas to understand before moving to components, props, state, hooks and APIs.

18. Quick Revision

React is a JavaScript library for building user interfaces.

React applications are built using reusable components.

SPA means Single Page Application.

Node.js is used by many React development tools.

npm manages JavaScript packages.

Vite can be used to create and run React projects.

JSX allows HTML-like syntax inside JavaScript.

JSX expressions use curly braces.

JSX tags must be properly closed.

className is used instead of class in JSX.

React state changes can cause a component to re-render.